hw 3

Jack

2026-10-06

```
library(dplyr); library(tidyr); library(tibble)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(readr); library(stringr); library(forcats); library(purrr)
```

Problem 1

```
sales_raw <- tibble(
  sale_id = 1:8,
  customer_id = c("C1", "C2", "C1", "C3", "C2", "C4", "C5", "C2"),
  item_raw = c(" Pen", "PEN ", "Notebook", " pen ",
               "FOLDER", "Folder ", " ERASER ", "pen"),
  amount_raw = c("$10.00", "$20.00", NA, "$15.00",
                 "$30.00", "$25.00", NA, "$20.00"),
  discount_raw = c("0%", "10%", "5%", "0%", "20%", "10%", "0%", "0%"),
  code_raw = c("ST-101", "ON-102", "ST-103", "ON-104",
               "bad", "ST-106", "ON-107X", "on-108"),
  tags = c("gift;urgent", "online", "gift", "member",
           "online;member", "gift", "member", "gift;member")
)
```

Problem A

```
sales_clean <- sales_raw %>%
  mutate(
    item_clean = item_raw %>% str_trim() %>% str_squish() %>% str_to_lower(),
    amount = parse_number(amount_raw),
    discount_rate = parse_number(discount_raw) / 100,
    net_amount = amount * (1 - discount_rate),
    size = case_when(
      is.na(net_amount) ~ "unrecorded",
      net_amount >= 20 ~ "large",
```

```
      TRUE ~ "small"
    )
  )

print(sales_clean %>% select(sale_id, item_clean, amount, discount_rate, net_amount, size))
```

```
## # A tibble: 8 x 6
##   sale_id item_clean amount discount_rate net_amount size
##     <int> <chr>       <dbl>         <dbl>      <dbl> <chr>
## 1       1 pen            10          0          10   small
## 2       2 pen            20          0.1        18   small
## 3       3 notebook       NA          0.05       NA   unrecorded
## 4       4 pen            15          0          15   small
## 5       5 folder         30          0.2        24   large
## 6       6 folder         25          0.1        22.5 large
## 7       7 eraser         NA          0          NA   unrecorded
## 8       8 pen            20          0          20   large
```

Part B

```
sales_codes <- sales_clean %>%
  mutate(
    code_clean = code_raw %>% str_trim() %>% str_to_upper(),
    valid_code = str_detect(code_clean, "^(ST|ON)-[0-9]{3}$")
  )

invalid_codes <- sales_codes %>%
  filter(!valid_code) %>%
  select(sale_id, code_raw, code_clean)
print(invalid_codes)
```

```
## # A tibble: 2 x 3
##   sale_id code_raw code_clean
##     <int> <chr>    <chr>
## 1       5 bad      BAD
## 2       7 ON-107X  ON-107X
```

```
valid_code_parts <- sales_codes %>%
  filter(valid_code) %>%
  select(sale_id, code_clean) %>%
  separate_wider_delim(code_clean, delim = "-", names = c("channel", "number"),
                       cols_remove = FALSE)
print(valid_code_parts)
```

```
## # A tibble: 6 x 4
##   sale_id channel number code_clean
##     <int> <chr>   <chr>  <chr>
## 1       1 ST      101    ST-101
## 2       2 ON      102    ON-102
## 3       3 ST      103    ST-103
## 4       4 ON      104    ON-104
## 5       6 ST      106    ST-106
## 6       8 ON      108    ON-108
```

```r
sale_tags <- sales_clean %>%
  select(sale_id, tags) %>%
  separate_longer_delim(tags, delim = ";")
print(sale_tags)
```

```
## # A tibble: 11 x 2
##    sale_id tags
##      <int> <chr>
##  1       1 gift
##  2       1 urgent
##  3       2 online
##  4       3 gift
##  5       4 member
##  6       5 online
##  7       5 member
##  8       6 gift
##  9       7 member
## 10       8 gift
## 11       8 member
```

```r
# separate_wider_delim() splits one column into several new columns while separate_longer_delim() split
```

Part C

```r
customer_summary <- sales_clean %>%
  summarise(
    sales_n = n(),
    known_n = sum(!is.na(net_amount)),
    large_n = sum(net_amount >= 20, na.rm = TRUE),
    large_share = if_else(known_n == 0, NA_real_, large_n / known_n),
    .by = customer_id
  ) %>%
  arrange(customer_id)
print(customer_summary)
```

```
## # A tibble: 5 x 5
##   customer_id sales_n known_n large_n large_share
##   <chr>         <int>   <int>   <int>       <dbl>
## 1 C1                2       1       0       0
## 2 C2                3       3       2       0.667
## 3 C3                1       1       0       0
## 4 C4                1       1       1       1
## 5 C5                1       0       0      NA
```

```r
# Numerator = 0.
# Denominator = 1.
# So C1's large_share = 0 / 1 = 0.

sales_factors <- sales_clean %>%
  mutate(
    item_factor = fct_infreq(factor(item_clean)),
    item_group = fct_lump_n(factor(item_clean), n = 2, other_level = "Other")
  )
print(levels(sales_factors$item_factor))
```

```
## [1] "pen"      "folder"   "eraser"   "notebook"
```

```
print(table(sales_factors$item_factor))
```

```
##
##      pen   folder   eraser notebook
##        4        2        1        1
```

```
print(levels(sales_factors$item_group))
```

```
## [1] "folder" "pen"    "Other"
```

```
print(table(sales_factors$item_group))
```

```
##
## folder    pen  Other
##      2      4      2
```

Problem 2

```
reports_raw <- tibble(
  form = c("A", "A", "A", "B", "B", "B"),
  line = c(1, 2, 3, 1, 2, 3),
  customer_id = c("C1", NA, "C2", "C3", NA, "C4"),
  round = c(1, 3, 1, 1, 2, 1),
  primary_amount = c(10, -999, 20, NA, 0, 30),
  backup_amount = c(NA, 15, 25, NA, 99, NA)
)

customers <- tibble(
  customer_id = c("C1", "C2", "C3", "C5"),
  region = c("North", "South", NA, "West")
)
```

Part A

```
reports_labeled <- reports_raw %>%
  arrange(form, line) %>%
  group_by(form) %>%
  fill(customer_id, .direction = "down") %>%
  ungroup()
print(reports_labeled)
```

```
## # A tibble: 6 x 6
##   form   line customer_id round primary_amount backup_amount
##   <chr> <dbl> <chr>       <dbl>          <dbl>         <dbl>
## 1 A         1 C1              1             10            NA
## 2 A         2 C1              3           -999            15
## 3 A         3 C2              1             20            25
## 4 B         1 C3              1             NA            NA
## 5 B         2 C3              2              0            99
## 6 B         3 C4              1             30            NA
```

```r
reports_clean <- reports_labeled %>%
  mutate(
    primary_clean = na_if(primary_amount, -999),
    amount = coalesce(primary_clean, backup_amount)
  )
print(reports_clean %>% select(form, line, customer_id, round, primary_amount, backup_amount, amount))
```

```
## # A tibble: 6 x 7
##   form   line customer_id round primary_amount backup_amount amount
##   <chr> <dbl> <chr>       <dbl>          <dbl>         <dbl>  <dbl>
## 1 A         1 C1              1             10            NA     10
## 2 A         2 C1              3           -999            15     15
## 3 A         3 C2              1             20            25     20
## 4 B         1 C3              1             NA            NA     NA
## 5 B         2 C3              2              0            99      0
## 6 B         3 C4              1             30            NA     30
```

```r
# coalesce() only moves on to backup_amount when primary_clean is NA.
# A recorded zero is a real, known value, so coalesce() keeps it instead of overwriting.
# If this wasn't in place a legitimate value of "0" would be replaced.

reports_for_complete <- reports_clean %>%
  select(customer_id, round, amount) %>%
  mutate(recorded_row = TRUE)

n_original <- nrow(reports_for_complete)
n_missing_before <- sum(is.na(reports_for_complete$amount))

# One missing amount already exists before completion as C3, round 1.
# An example of an absent customer-round combination that isn't in the table is C1, round 2.

reports_complete <- reports_for_complete %>%
  complete(customer_id, round = 1:3, fill = list(recorded_row = FALSE))

n_total_after <- nrow(reports_complete)
n_added <- n_total_after - n_original
n_missing_after <- sum(is.na(reports_complete$amount))

cat("Original rows:", n_original, "\n")
```

```
## Original rows: 6
```

```r
cat("Added rows:", n_added, "\n")
```

```
## Added rows: 6
```

```r
cat("Missing amounts before completion:", n_missing_before, "\n")
```

```
## Missing amounts before completion: 1
```

```r
cat("Missing amounts after completion:", n_missing_after, "\n")
```

```
## Missing amounts after completion: 7
```

```r
print(reports_complete)
```

```
## # A tibble: 12 x 4
```

```
##    customer_id round amount recorded_row
##    <chr>       <dbl>  <dbl> <lgl>
##  1 C1              1     10 TRUE
##  2 C1              2     NA FALSE
##  3 C1              3     15 TRUE
##  4 C2              1     20 TRUE
##  5 C2              2     NA FALSE
##  6 C2              3     NA FALSE
##  7 C3              1     NA TRUE
##  8 C3              2      0 TRUE
##  9 C3              3     NA FALSE
## 10 C4              1     30 TRUE
## 11 C4              2     NA FALSE
## 12 C4              3     NA FALSE
```

Part B

```
cat("\nDuplicate customer IDs in customers:", anyDuplicated(customers$customer_id), "\n")
```

```
##
## Duplicate customer IDs in customers: 0
```

```
cat("Missing customer IDs in customers:", sum(is.na(customers$customer_id)), "\n")
```

```
## Missing customer IDs in customers: 0
```

```
dup_customer_rounds <- reports_clean %>% count(customer_id, round) %>% filter(n > 1)

print(dup_customer_rounds)
```

```
## # A tibble: 0 x 3
## # i 3 variables: customer_id <chr>, round <dbl>, n <int>
```

```
customers_marked <- customers %>% mutate(customer_found = TRUE)

reports_joined <- reports_clean %>%
  left_join(customers_marked, by = join_by(customer_id),
            relationship = "many-to-one", na_matches = "never") %>%
  mutate(customer_found = coalesce(customer_found, FALSE))
print(reports_joined %>% select(customer_id, round, amount, region, customer_found))
```

```
## # A tibble: 6 x 5
##   customer_id round amount region customer_found
##   <chr>       <dbl>  <dbl> <chr>  <lgl>
## 1 C1              1     10 North  TRUE
## 2 C1              3     15 North  TRUE
## 3 C2              1     20 South  TRUE
## 4 C3              1     NA <NA>   TRUE
## 5 C3              2      0 <NA>   TRUE
## 6 C4              1     30 <NA>   FALSE
```

```
only_in_reports <- anti_join(reports_clean, customers, by = "customer_id", na_matches = "never") %>%
  distinct(customer_id)
only_in_customers <- anti_join(customers, reports_clean, by = "customer_id", na_matches = "never") %>%
  distinct(customer_id)
print(only_in_reports)
```

```
## # A tibble: 1 x 1
##   customer_id
##   <chr>
## 1 C4
```

```
print(only_in_customers)
```

```
## # A tibble: 1 x 1
##   customer_id
##   <chr>
## 1 C5
```

```
# C4 appears only in reports_clean, and 5 appears only in customers.
# The region for C3 is NA, but customer_found = TRUE tells us C3 is in the customer master file.
# C4 customer_found = FALSE tells us the NA region is not a blank field, but a complete absence of any
```

Part C

```
# Because customers_bad has two rows for C1, each of C1's two reports matches both C1 rows and is dupli
# C1's original amounts get counted twice, which inflates the ignoring-NA total from 75 to 100 .


# With relationship = "many-to-one", dplyr would raise an error.
# This is because that setting asserts each report row matches at most one customer row, and C1 matches
# Allowing "many-to-many" does not resolve the conflicting region values for C1 - it just silences the

reports_wide <- reports_complete %>%
  select(customer_id, round, amount) %>%
  pivot_wider(names_from = round, values_from = amount, names_prefix = "round_")
print(reports_wide)
```

```
## # A tibble: 4 x 4
##   customer_id round_1 round_2 round_3
##   <chr>         <dbl>   <dbl>   <dbl>
## 1 C1               10      NA      15
## 2 C2               20      NA      NA
## 3 C3               NA       0      NA
## 4 C4               30      NA      NA
```

```
long_back <- reports_wide %>%
  pivot_longer(cols = starts_with("round_"), names_to = "round",
               values_to = "amount", values_drop_na = TRUE)
cat("\nRows after pivot back with values_drop_na = TRUE:", nrow(long_back), "\n")
```

```
##
## Rows after pivot back with values_drop_na = TRUE: 5
```

```
# Prediction: 5 rows. Of the 12 customer-round combinations in reports_complete, only 5 have a known am

# values_drop_na = TRUE  removes rows with NA after reshaping it. It doesn't have any way to recover an
```

Problem 3

Part A

```r
two_sample_t <- function(x, y, equal_var = FALSE, alpha = 0.05) {
  nx <- length(x); ny <- length(y)
  mean_x <- mean(x); mean_y <- mean(y)
  var_x <- var(x); var_y <- var(y)

  if (equal_var) {
    sp2 <- ((nx - 1) * var_x + (ny - 1) * var_y) / (nx + ny - 2)
    se <- sqrt(sp2 * (1 / nx + 1 / ny))
    df <- nx + ny - 2
    method <- "Pooled"
  } else {
    se <- sqrt(var_x / nx + var_y / ny)
    df <- (var_x / nx + var_y / ny)^2 /
      ((var_x / nx)^2 / (nx - 1) + (var_y / ny)^2 / (ny - 1))
    method <- "Welch"
  }

  t_stat <- (mean_x - mean_y) / se
  p_value <- 2 * pt(abs(t_stat), df = df, lower.tail = FALSE)
  reject <- p_value < alpha

  list(method = method, t_stat = t_stat, df = df, p_value = p_value, reject = reject)
}

x_check <- c(12, 15, 14, 10, 13)
y_check <- c(9, 11, 8, 10, 7, 12, 9, 8)

ref_welch <- t.test(x_check, y_check, var.equal = FALSE)
ref_pooled <- t.test(x_check, y_check, var.equal = TRUE)

welch_result <- two_sample_t(x_check, y_check)
pooled_result <- two_sample_t(x_check, y_check, equal_var = TRUE)
print(welch_result)
```

```
## $method
## [1] "Welch"
##
## $t_stat
## [1] 3.403072
##
## $df
## [1] 7.678614
##
## $p_value
## [1] 0.009907123
##
## $reject
## [1] TRUE
```

```r
print(pooled_result)
```

```
## $method
## [1] "Pooled"
##
## $t_stat
## [1] 3.526436
##
## $df
## [1] 11
##
## $p_value
## [1] 0.00474447
##
## $reject
## [1] TRUE
```

```r
welch_diffs <- c(
  t_stat = abs(welch_result$t_stat - unname(ref_welch$statistic)),
  df     = abs(welch_result$df - unname(ref_welch$parameter)),
  p_value = abs(welch_result$p_value - ref_welch$p.value)
)
pooled_diffs <- c(
  t_stat = abs(pooled_result$t_stat - unname(ref_pooled$statistic)),
  df     = abs(pooled_result$df - unname(ref_pooled$parameter)),
  p_value = abs(pooled_result$p_value - ref_pooled$p.value)
)
print(welch_diffs)
```

```
##       t_stat           df      p_value
## 4.440892e-16 2.664535e-15 5.204170e-18
```

```r
print(pooled_diffs)
```

```
##  t_stat      df p_value
##       0       0       0
```

```r
cat("All differences smaller than 1e-8:", all(c(welch_diffs, pooled_diffs) < 1e-8), "\n")
```

```
## All differences smaller than 1e-8: TRUE
```

Part B

```r
B <- 1000
n_x <- 20
n_y <- 40
alpha <- 0.05

scenarios <- data.frame(
  scenario = c("equal_null", "unequal_null",
               "equal_shift", "unequal_shift"),
  mean_x = c(0, 0, 0, 0),
  mean_y = c(0, 0, 1, 1),
  sd_x = c(1, 3, 1, 3),
  sd_y = c(1, 1, 1, 1)
```

```
)

# Run once immediately before the simulation, not inside a loop.
set.seed(2026)
```

```
n_scenarios <- nrow(scenarios)
total_rows <- n_scenarios * B * 2

scenario_vec <- character(total_rows)
replicate_vec <- integer(total_rows)
method_vec <- character(total_rows)
t_stat_vec <- numeric(total_rows)
df_vec <- numeric(total_rows)
p_value_vec <- numeric(total_rows)
reject_vec <- logical(total_rows)

row_i <- 1
for (s in 1:n_scenarios) {
  for (b in 1:B) {
    x <- rnorm(n_x, mean = scenarios$mean_x[s], sd = scenarios$sd_x[s])
    y <- rnorm(n_y, mean = scenarios$mean_y[s], sd = scenarios$sd_y[s])

    res_welch <- two_sample_t(x, y, equal_var = FALSE, alpha = alpha)
    scenario_vec[row_i] <- scenarios$scenario[s]
    replicate_vec[row_i] <- b
    method_vec[row_i] <- res_welch$method
    t_stat_vec[row_i] <- res_welch$t_stat
    df_vec[row_i] <- res_welch$df
    p_value_vec[row_i] <- res_welch$p_value
    reject_vec[row_i] <- res_welch$reject
    row_i <- row_i + 1

    res_pooled <- two_sample_t(x, y, equal_var = TRUE, alpha = alpha)
    scenario_vec[row_i] <- scenarios$scenario[s]
    replicate_vec[row_i] <- b
    method_vec[row_i] <- res_pooled$method
    t_stat_vec[row_i] <- res_pooled$t_stat
    df_vec[row_i] <- res_pooled$df
    p_value_vec[row_i] <- res_pooled$p_value
    reject_vec[row_i] <- res_pooled$reject
    row_i <- row_i + 1
  }
}

simulation_results <- tibble(
  scenario = scenario_vec,
  replicate = replicate_vec,
  method = method_vec,
  t_stat = t_stat_vec,
  df = df_vec,
  p_value = p_value_vec,
  reject = reject_vec
)
```

```r
print(head(simulation_results))
```

```
## # A tibble: 6 x 7
##   scenario   replicate method t_stat    df p_value reject
##   <chr>          <int> <chr>   <dbl> <dbl>   <dbl> <lgl>
## 1 equal_null         1 Welch  -1.64   36.3  0.109  FALSE
## 2 equal_null         1 Pooled -1.67   58    0.0999 FALSE
## 3 equal_null         2 Welch   0.450  43.9  0.655  FALSE
## 4 equal_null         2 Pooled  0.427  58    0.671  FALSE
## 5 equal_null         3 Welch   1.58   30.8  0.123  FALSE
## 6 equal_null         3 Pooled  1.72   58    0.0903 FALSE
```

```r
cat("Total rows in simulation_results:", nrow(simulation_results), "\n")
```

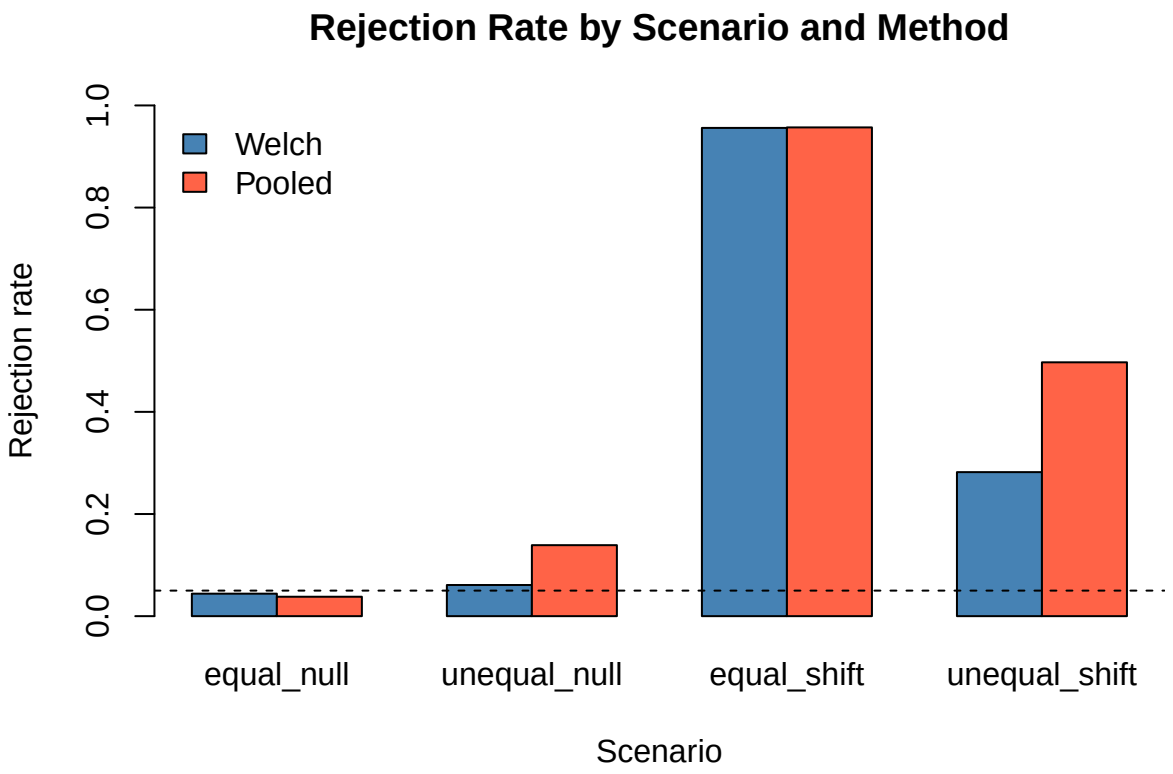
```
## Total rows in simulation_results: 8000
```

```r
simulation_summary <- simulation_results %>%
  summarise(
    mean_t = round(mean(t_stat), 3),
    sd_t = round(sd(t_stat), 3),
    rejection_rate = round(mean(reject), 3),
    .by = c(scenario, method)
  )
print(simulation_summary)
```

```
## # A tibble: 8 x 5
##   scenario      method mean_t  sd_t rejection_rate
##   <chr>         <chr>   <dbl> <dbl>          <dbl>
## 1 equal_null    Welch  -0.019  1.04          0.044
## 2 equal_null    Pooled -0.018  1.02          0.038
## 3 unequal_null  Welch   0.036  1.08          0.061
## 4 unequal_null  Pooled  0.048  1.40          0.139
## 5 equal_shift   Welch  -3.77   1.12          0.956
## 6 equal_shift   Pooled -3.76   1.08          0.957
## 7 unequal_shift Welch  -1.54   1.05          0.282
## 8 unequal_shift Pooled -2.02   1.36          0.497
```

```r
plot_data <- simulation_summary %>%
  select(scenario, method, rejection_rate) %>%
  pivot_wider(names_from = method, values_from = rejection_rate)
plot_matrix <- t(as.matrix(plot_data[, c("Welch", "Pooled")]))
colnames(plot_matrix) <- plot_data$scenario

barplot(plot_matrix, beside = TRUE, ylim = c(0, 1),
        col = c("steelblue", "tomato"),
        main = "Rejection Rate by Scenario and Method",
        ylab = "Rejection rate", xlab = "Scenario")
abline(h = 0.05, lty = 2)
legend("topleft", legend = c("Welch", "Pooled"), fill = c("steelblue", "tomato"), bty = "n")
```




```
# Under both null scenarios Welch's rejection  rate stays close to 0.05 regardless of whether the popul
# The Type I error is well controlled and the pooled test is also close to 0.05 when variances are equa
# When the  means differ by one unit, both tests' power rises far above 0.05.
# The test rejects quite often and this only helps if its Type I error rate is also controlled. So resu
```